

RETAILMART V3 • SQL

Data Quality, Deduplication & Cleansing

WhatsApp Announcement

Data Quality, Deduplication & Cleansing

Real data is messy. Duplicate rows, NULL clusters, malformed phone numbers, suspicious outliers. Today you learn the SQL patterns to FIND and FIX these problems -- the skills that separate a junior analyst from a senior one.

Part 1: Finding and Removing Duplicates

Part 2: NULL Strategies and Anomaly Detection

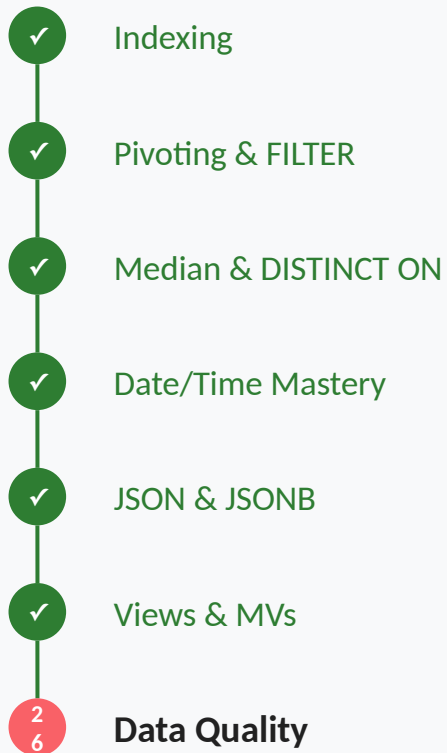
Part 3: Regex and Standardization

This is the last day of new concepts before the capstone. After today you have every SQL tool you need to build the full analytics layer in the capstone.

TODAY'S AGENDA (2 Hours)

Section	Time	Topics
Part 1: Deduplication	2 Hours	Finding duplicates with GROUP BY HAVING, Deduplication with ROW_NUMBER, Keeping the right record
Part 2: NULLs and Anomalies		COALESCE, NULLIF, IS DISTINCT FROM, NULL cluster detection, Threshold-based anomaly flagging
Part 3: Regex & Standardization		Pattern matching with ~ and ~*, REGEXP_REPLACE for cleanup, LOWER, TRIM for normalization

YOUR SQL JOURNEY



← HERE

What We Covered Yesterday

- CREATE VIEW for reusable query logic
- Layered views: raw -> cleaned -> enriched -> KPIs
- Materialized Views for precomputed, fast results
- REFRESH CONCURRENTLY with unique index requirement

The Duplicate Customer Problem

A customer signed up twice with the same email. Now their orders are split across two customer_ids. Your revenue-per-customer analysis is wrong. Your churn model thinks one active customer is two lapsed ones. Deduplication is step one of any data quality audit.

Find Duplicates

```
-- Find duplicate emails:
SELECT
    email,
    COUNT(*) AS duplicate_count
FROM customers.customers
GROUP BY email
HAVING COUNT(*) > 1
ORDER BY duplicate_count DESC;
```

This finds emails with more than one customer record. But which record should you keep?

Dedup with ROW_NUMBER

```
-- Keep the most recently active record per email:
WITH ranked AS (
  SELECT
    *,
    ROW_NUMBER() OVER (
      PARTITION BY email
      ORDER BY registration_date DESC NULLS LAST
    ) AS rn
  FROM customers.customers
)
-- rn = 1 is the record to KEEP
-- rn > 1 are the duplicates

-- View deduplicated records (rn = 1 keeps, rn > 1 discards):
SELECT * FROM ranked WHERE rn = 1;
-- (to see the duplicates instead, change to: WHERE rn > 1)
```

The ROW_NUMBER Dedup Pattern

- PARTITION BY the column(s) that define 'same' (email, phone, etc.)
- ORDER BY the column that determines which record to keep (most recent, highest ID, etc.)
- rn = 1 is the keeper. rn > 1 are duplicates.
- This pattern works in every SQL database, not just PostgreSQL.

PRO TIP

PRO TIP

NULLS LAST in the ORDER BY ensures that records with NULL updated_at do not accidentally win. By default, NULLs sort first in DESC order, which would keep the wrong record.

Practice 1: Find Duplicate Customers by Email

EASY

SCENARIO: The data team suspects duplicate customer records.

TASK: Find all emails that appear more than once in `customers.customers`. Show the email, count of duplicates, and the `customer_ids` as a comma-separated list.

HINT: GROUP BY email, HAVING COUNT(*) > 1. Use STRING_AGG for the ID list.

Answer

✓ ANSWER

```
SELECT
    email,
    COUNT(*) AS dup_count,
    STRING_AGG(
        customer_id::TEXT, ', '
        ORDER BY customer_id
    ) AS customer_ids
FROM customers.customers
GROUP BY email
HAVING COUNT(*) > 1
ORDER BY dup_count DESC;
```

Practice 2: Deduplicate Keeping Most Recent

MEDIUM

SCENARIO: For each duplicate email, keep only the record with the most recent activity (highest customer_id as a proxy for most recent).

TASK: Use ROW_NUMBER to assign ranks within each email group. Show: customer_id, email, rn, and a 'keep' or 'discard' label.

Answer (1/2)

✓ ANSWER

```
WITH ranked AS (  
  SELECT  
    customer_id,  
    email,  
    first_name,  
    last_name,  
    ROW_NUMBER() OVER (  
      PARTITION BY email  
      ORDER BY customer_id DESC  
    ) AS rn  
  FROM customers.customers  
)  
SELECT  
  customer_id,  
  email,  
  first_name || ' ' || last_name AS name,  
  rn,  
  CASE WHEN rn = 1 THEN 'KEEP'  
        ELSE 'DISCARD' END AS action  
FROM ranked  
WHERE email IN (  
  SELECT email FROM customers.customers
```

Answer (2/2)



ANSWER

```
GROUP BY email HAVING COUNT(*) > 1  
)  
ORDER BY email, rn;
```

The Missing Phone Numbers

30% of your customers have NULL phone numbers. Is that a data quality issue or normal? If NULL clusters exist -- all from one signup source or one date range -- that points to a broken import pipeline. Finding NULL PATTERNS is more important than just counting NULLs.

NULL Handling Functions

-- COALESCE

```
COALESCE(col, 'default')  
-- First non-NULL value
```

-- NULLIF

```
NULLIF(a, b)  
-- Returns NULL if a = b, else a  
-- Perfect for division by zero
```

-- IS DISTINCT FROM

```
a IS DISTINCT FROM b  
-- NULL-safe comparison  
-- NULL IS DISTINCT FROM NULL = false
```

Find NULL Patterns

```
-- Which columns have the most NULLs?
```

```
SELECT
    COUNT(*) AS total_rows,
    COUNT(*) - COUNT(phone) AS null_phone,
    COUNT(*) - COUNT(email) AS null_email,
    ROUND((COUNT(*) - COUNT(phone))
        * 100.0 / COUNT(*), 1) AS null_phone_pct
FROM customers.customers;
```

```
-- Do NULLs cluster by signup date?
```

```
SELECT
    DATE_TRUNC('month', registration_date)::DATE AS month,
    COUNT(*) AS total,
    COUNT(*) - COUNT(phone) AS null_phone,
    ROUND((COUNT(*) - COUNT(phone))
        * 100.0 / COUNT(*), 1) AS null_pct
FROM customers.customers
GROUP BY DATE_TRUNC('month', registration_date)
ORDER BY month;
```

Anomaly Flagging

```
-- Flag orders with suspiciously high amounts:
SELECT
  order_id,
  net_total,
  CASE
    WHEN net_total > 500000 THEN 'SUSPICIOUS'
    WHEN net_total < 10      THEN 'TOO_LOW'
    ELSE 'normal'
  END AS flag
FROM sales.orders
WHERE net_total > 500000
      OR net_total < 10;
```

Price Jump Detection (1/2)

```
-- Flag products with > 50% price jump:
WITH price_changes AS (
  SELECT
    oi.prod_id,
    p.product_name,
    oi.unit_price,
    LAG(oi.unit_price) OVER (
      PARTITION BY oi.prod_id
      ORDER BY o.order_date
    ) AS prev_price
  FROM sales.order_items oi
  JOIN sales.orders o ON oi.order_id = o.order_id
  JOIN products.products p
    ON oi.prod_id = p.product_id
)
SELECT prod_id, product_name,
       prev_price, unit_price,
       ROUND((unit_price - prev_price)
         / NULLIF(prev_price, 0) * 100, 1) AS pct_change
FROM price_changes
```

Price Jump Detection (2/2)

```
WHERE prev_price IS NOT NULL
      AND ABS(unit_price - prev_price)
          / NULLIF(prev_price, 0) > 0.50
ORDER BY pct_change DESC;
```

Practice 3: Business Threshold Validation

HARD

SCENARIO: The finance team defines business rules: no order should exceed 500,000 and no order should have a `net_total` of 0 or negative.

TASK: Find all orders that violate these thresholds. Show `order_id`, `net_total`, `order_status`, `order_date`, and the violation type.

Answer

✓ ANSWER

```
SELECT
  order_id,
  net_total,
  order_status,
  order_date,
  CASE
    WHEN net_total > 500000
      THEN 'exceeds_max'
    WHEN net_total <= 0
      THEN 'zero_or_negative'
  END AS violation
FROM sales.orders
WHERE net_total > 500000
      OR net_total <= 0
ORDER BY net_total DESC;
```

The Phone Number Mess

Customer phone numbers come in every format: +91-9876543210, 09876543210, 9876543210, (098) 7654-3210. To match customers across systems, you need a single canonical format. Regex + REGEXP_REPLACE turns the mess into clean, consistent data.

Regex Operators (1/2)

```
-- ~ : case-sensitive regex match
SELECT * FROM customers.customers
WHERE email ~ '^[a-z]+@gmail\.com$';
```

```
-- ~* : case-insensitive regex match
SELECT * FROM customers.customers
WHERE email ~* '^[a-z]+@gmail\.com$';
```

```
-- !~ : does NOT match
SELECT * FROM customers.customers
WHERE phone !~ '^\+91';
```

```
-- Common patterns:
-- ^      start of string
-- $      end of string
-- [0-9]   any digit
-- [a-z]   any lowercase letter
-- .      any character
-- +      one or more
-- *      zero or more
```

Regex Operators (2/2)

```
-- \d      digit (same as [0-9])
```

Phone Number Standardization

```
-- Strip everything except digits:
```

```
SELECT
    phone AS original,
    REGEXP_REPLACE(phone, '[^0-9]', '', 'g')
    AS digits_only
FROM customers.customers
WHERE phone IS NOT NULL
LIMIT 10;
```

```
-- Standardize to +91XXXXXXXXXX format:
```

```
SELECT
    phone AS original,
    '+91' || RIGHT(
        REGEXP_REPLACE(phone, '[^0-9]', '', 'g'),
        10
    ) AS standardized
FROM customers.customers
WHERE phone IS NOT NULL;
```

Name and Text Cleanup

```
-- Standardize for matching:
SELECT
    LOWER(TRIM(first_name))      AS clean_first,
    LOWER(TRIM(last_name))      AS clean_last,
    LOWER(TRIM(email))          AS clean_email
FROM customers.customers;

-- Find malformed emails:
SELECT email
FROM customers.customers
WHERE email !~ '^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$';
```

Common Data Quality Mistakes

Deduping without choosing the right keeper: Always define which record to keep (most recent, most complete, highest ID). Random dedup loses data.

Ignoring NULL patterns: Counting NULLs is not enough. Check if they cluster by date, source, or category to find broken pipelines.

Case-sensitive matching: LOWER + TRIM before comparing strings. 'John' and 'john' and ' John ' should all match.

Over-cleaning: Do not modify the source data. Apply cleaning in views or CTEs. Keep raw data intact for audit.

Practice 4: Standardize Phone Numbers

MEDIUM

SCENARIO: Customer phone numbers are inconsistent. Standardize them for matching.

TASK: Show original phone and standardized phone (digits only, last 10 digits, prefixed with +91). Flag phones that do not have exactly 10 digits after stripping.

Answer



ANSWER

```
SELECT
  phone AS original,
  REGEXP_REPLACE(phone, '^[0-9]', '', 'g')
  AS digits_only,
  '+91' || RIGHT(
    REGEXP_REPLACE(phone, '^[0-9]', '', 'g'),
    10) AS standardized,
  CASE
    WHEN LENGTH(REGEXP_REPLACE(
      phone, '^[0-9]', '', 'g')) < 10
      THEN 'TOO_SHORT'
    WHEN LENGTH(REGEXP_REPLACE(
      phone, '^[0-9]', '', 'g')) > 12
      THEN 'TOO_LONG'
    ELSE 'OK'
  END AS validation
FROM customers.customers
WHERE phone IS NOT NULL
ORDER BY validation DESC;
```

Practice 5: Backfill NULLs with COALESCE

HARD

SCENARIO: A customer's city comes from their address (`customers.addresses`). If a customer has no address on file, fall back to the city of their most recent order's store.

TASK: For each customer, resolve a city: use the address city, else the most recent order's store city. Show `customer_id`, `address_city`, and the resolved (backfilled) city.

Answer



ANSWER

```
WITH latest_store AS (  
    SELECT DISTINCT ON (cust_id)  
        cust_id,  
        store_id  
    FROM sales.orders  
    ORDER BY cust_id, order_date DESC  
)  
SELECT  
    c.customer_id,  
    a.city AS address_city,  
    COALESCE(a.city, s.city) AS backfilled_city  
FROM customers.customers c  
LEFT JOIN customers.addresses a  
    ON a.customer_id = c.customer_id  
LEFT JOIN latest_store ls  
    ON c.customer_id = ls.cust_id  
LEFT JOIN stores.stores s  
    ON ls.store_id = s.store_id  
WHERE a.city IS NULL  
ORDER BY c.customer_id;
```

Practice 6: Full Data Quality Audit

CRAZY

SCENARIO: Run a comprehensive data quality audit on customers.customers.

TASK: Build a single query that reports: total rows, duplicate emails count, NULL phone count, NULL email count, malformed emails count, and avg NULLs per row. Present as a single-row summary.

Answer

✓ ANSWER

```
SELECT
  COUNT(*) AS total_rows,
  (SELECT COUNT(*) FROM (
    SELECT email FROM customers.customers
    GROUP BY email HAVING COUNT(*) > 1
  ) d) AS duplicate_emails,
  COUNT(*) - COUNT(phone)
  AS null_phone,
  COUNT(*) - COUNT(email)
  AS null_email,
  COUNT(*) FILTER (WHERE email !~
    '^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$')
  AS malformed_emails,
  ROUND((
    (COUNT(*) - COUNT(phone))
    + (COUNT(*) - COUNT(email))
  )::NUMERIC / NULLIF(COUNT(*), 0), 2)
  AS avg_nulls_per_row
FROM customers.customers;
```

Q: How do you find and remove duplicates in SQL?

A: Two steps: (1) Find duplicates with GROUP BY + HAVING COUNT(*) > 1. (2) Deduplicate with ROW_NUMBER() OVER (PARTITION BY duplicate_key ORDER BY tiebreaker), keeping rn = 1 as the canonical record. The ORDER BY determines which record survives -- typically the most recent or most complete.

Q: What is IS DISTINCT FROM?

A: A NULL-safe equality check. Regular = returns NULL when either side is NULL (which evaluates to false in WHERE). IS DISTINCT FROM treats NULL as a value: NULL IS DISTINCT FROM NULL returns false (they are the same), NULL IS DISTINCT FROM 5 returns true. Use it when comparing columns that may contain NULLs.

Q: How do you handle data quality in SQL?

A: A systematic approach: (1) Profile: count NULLs, check cardinality, find duplicates. (2) Validate: check against business rules (ranges, patterns, referential integrity). (3) Clean: standardize with LOWER/TRIM/REGEXP_REPLACE. (4) Deduplicate with ROW_NUMBER. (5) Monitor: create views or queries that flag new quality issues automatically.

HW 1: NULL Profile

EASY

TASK: For the products.products table, count NULLs in each column. Show column name, null count, and null percentage.

Answer



ANSWER

```
SELECT
    'supplier_id' AS column_name,
    COUNT(*) - COUNT(supplier_id) AS nulls,
    ROUND((COUNT(*) - COUNT(supplier_id))
        * 100.0 / COUNT(*), 1) AS null_pct
FROM products.products
UNION ALL
SELECT 'brand_id',
    COUNT(*) - COUNT(brand_id),
    ROUND((COUNT(*) - COUNT(brand_id))
        * 100.0 / COUNT(*), 1)
FROM products.products
UNION ALL
SELECT 'cost_price',
    COUNT(*) - COUNT(cost_price),
    ROUND((COUNT(*) - COUNT(cost_price))
        * 100.0 / COUNT(*), 1)
FROM products.products
ORDER BY nulls DESC;
```

HW 2: Dedup Orders by Customer+Date

MEDIUM

TASK: Find cases where the same customer placed multiple orders on the same date. Use ROW_NUMBER to identify which to keep (highest net_total). Show duplicates with their rank.

Answer



ANSWER

```
WITH ranked AS (  
    SELECT order_id, cust_id,  
           order_date::DATE AS day,  
           net_total,  
           ROW_NUMBER() OVER (  
               PARTITION BY cust_id, order_date::DATE  
               ORDER BY net_total DESC  
           ) AS rn  
    FROM sales.orders  
)  
SELECT *,  
       CASE WHEN rn = 1 THEN 'KEEP' ELSE 'DUP' END AS action  
FROM ranked  
WHERE (cust_id, day) IN (  
    SELECT cust_id, order_date::DATE  
    FROM sales.orders  
    GROUP BY cust_id, order_date::DATE  
    HAVING COUNT(*) > 1  
)  
ORDER BY cust_id, day, rn;
```

HW 3: Find Malformed Emails

MEDIUM

TASK: Find all customers with emails that do not match a basic email pattern. Show customer_id, email, and what looks wrong.

Answer

 ANSWER

```
SELECT
  customer_id,
  email,
  CASE
    WHEN email !~ '@' THEN 'missing_at_sign'
    WHEN email !~ '\.' THEN 'missing_dot'
    WHEN email ~ '\s' THEN 'contains_whitespace'
    WHEN email ~ '@.*@' THEN 'multiple_at_signs'
    ELSE 'other_issue'
  END AS issue
FROM customers.customers
WHERE email !~
  '^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'
ORDER BY issue, email;
```

HW 4: Reconcile Two Product Lists

HARD

TASK: Two sources have product names with different casing and spacing. Match them by `LOWER(TRIM(product_name))`. Show matched pairs and unmatched products from each source.

Answer

✓ ANSWER

```
-- Simulate: match products across two tables
-- using normalized names:
WITH norm_products AS (
    SELECT product_id, product_name,
           LOWER(TRIM(product_name)) AS norm_name
    FROM products.products
)
SELECT
    norm_name,
    COUNT(*) AS matches,
    STRING_AGG(product_name, ' | '
               ORDER BY product_id) AS variations
FROM norm_products
GROUP BY norm_name
HAVING COUNT(*) > 1
ORDER BY matches DESC;
```

HW 5: Price Anomaly Report

HARD

TASK: Detect products whose price jumps more than 50% between consecutive orders. Show product_id, product_name, old_price, new_price, pct_change, and flag as 'DATA_ERROR'.

Answer (1/2)

✓ ANSWER

```
WITH prices AS (  
    SELECT oi.prod_id,  
           p.product_name,  
           oi.unit_price,  
           o.order_date,  
           LAG(oi.unit_price) OVER (  
               PARTITION BY oi.prod_id  
               ORDER BY o.order_date  
           ) AS prev_price  
    FROM sales.order_items oi  
    JOIN sales.orders o ON oi.order_id = o.order_id  
    JOIN products.products p  
      ON oi.prod_id = p.product_id  
)  
SELECT prod_id, product_name,  
       prev_price, unit_price,  
       ROUND((unit_price - prev_price)  
             / NULLIF(prev_price, 0) * 100, 1)  
       AS pct_change,  
       'DATA_ERROR' AS flag  
FROM prices  
WHERE prev_price IS NOT NULL
```

Answer (2/2)

 ANSWER

```
AND ABS(unit_price - prev_price)
    / NULLIF(prev_price, 0) > 0.50
ORDER BY ABS(unit_price - prev_price)
    / NULLIF(prev_price, 0) DESC;
```

HW 6: Complete Data Quality Dashboard

CRAZY

TASK: Build a data quality dashboard view that runs daily. For each major table (customers, orders, products), show: row_count, duplicate_count, null_cluster_count, anomaly_count. Present as one row per table.

Answer (1/2)

✓ ANSWER

```
SELECT 'customers' AS tbl,
      (SELECT COUNT(*) FROM customers.customers)
      AS rows,
      (SELECT COUNT(*) FROM (
        SELECT email FROM customers.customers
        GROUP BY email HAVING COUNT(*) > 1) x)
      AS dup_emails,
      (SELECT COUNT(*) FROM customers.customers
        WHERE phone IS NULL) AS null_phones,
      0 AS anomalies
UNION ALL
SELECT 'orders',
      (SELECT COUNT(*) FROM sales.orders),
      0,
      (SELECT COUNT(*) FROM sales.orders
        WHERE net_total IS NULL),
      (SELECT COUNT(*) FROM sales.orders
        WHERE net_total > 500000
        OR net_total <= 0)
UNION ALL
SELECT 'products',
      (SELECT COUNT(*) FROM products.products),
```

Answer (2/2)

✓ ANSWER

```
0,  
(SELECT COUNT(*) FROM products.products  
  WHERE supplier_id IS NULL),  
(SELECT COUNT(*) FROM products.products  
  WHERE price <= 0);
```

KEY TAKEAWAYS (1/2)

GROUP BY HAVING finds duplicates: GROUP BY the key, HAVING COUNT(*) > 1 shows which groups have duplicates.

ROW_NUMBER deduplicates: PARTITION BY duplicate_key, ORDER BY tiebreaker. rn = 1 is the keeper.

Look for NULL patterns, not just counts: NULLs that cluster by date or source indicate a pipeline bug, not random missing data.

Regex with ~ and REGEXP_REPLACE: ~ for matching, REGEXP_REPLACE for cleanup. 'g' flag for global replacement.

LOWER + TRIM before comparing:

KEY TAKEAWAYS (2/2)

Always normalize text before matching or deduplicating.

Clean in views, not in source tables: Apply all cleaning in CTEs or views. Keep raw data intact for audit.

earlier Complete!

You now have every SQL pattern an analyst needs: window functions, query optimization, pivoting, percentiles, time-series, JSON, views, and data quality. Next week is Cohort/RFM analysis (Cohort & RFM), Interview Prep (Interview Prep), and the 2-day Capstone (the capstone) where you build a full analytics dashboard.